

Implementación de un Algoritmo para el Diagnóstico de Salud en Fresadoras Industriales

Francisco Marín Castillo
Ingeniería en inteligencia artificial
Universidad Iberoamericana León
León, México
192405-1@iberoleon.edu.mx

Diego Mares Rodriguez
Ingeniería en inteligencia artificial
Universidad Iberoamericana León
León, México
192218-1@iberoleon.edu.mx

1. Definición del Problema

La industria manufacturera moderna enfrenta pérdidas económicas significativas debido a paradas no planificadas en las líneas de producción, ocasionadas por fallas imprevistas en la maquinaria. El problema específico a abordar es la **predicción anticipada de fallas** en equipos rotativos (motores/fresadoras) para transicionar de un mantenimiento correctivo o preventivo, a un enfoque predictivo. El problema se define como una tarea de **Clasificación Binaria Supervisada**. El objetivo del modelo es aprender una función de mapeo y que determine la probabilidad de falla de una máquina en un ciclo operativo dado:

- **Variables de Entrada (X):** Se utilizarán lecturas de sensores simulados que reflejan las condiciones operativas de la máquina:
 - **Temperatura del aire [K]:** Condiciones ambientales.
 - **Temperatura del proceso [K]:** Calor generado por la fricción y operación.
 - **Velocidad de rotación [rpm]:** Frecuencia de giro del husillo.
 - **Torque [Nm]:** Fuerza de torsión aplicada.
 - **Desgaste de la herramienta [min]:** Tiempo acumulado de uso de la herramienta de corte.
 - **Etc.**
- **Variable de Salida (Target / y):** Variable categórica binaria que indica el estado de la máquina:
 - **0:** Operación Normal.
 - **1:** Falla Inminente (incluyendo fallo por disipación de calor, fallo de potencia, fallo por sobreesfuerzo, etc.).

2. Descripción de la Base de Datos

Para el desarrollo del modelo predictivo se utiliza el conjunto de datos “*AI4I 2020 Predictive Maintenance Dataset*”, obtenido del Repositorio de Machine Learning de la UCI(Universidad de California)

- **Dimensiones:** El dataset consta de **10,000 instancias** (filas) y **14 atributos** (columnas).
- **Variables de Entrada (Features):** Se seleccionaron variables físicas críticas para el monitoreo de condición.
- **Variable Objetivo (Target):** La columna “Machine Failure (int64)” actúa como etiqueta binaria, donde '0' indica operación normal y '1' indica evento de falla.
- **Justificación del Dataset:** El análisis exploratorio revela una distribución de clases de **9,661 casos negativos (Normal)** frente a **339 casos positivos (Falla)**. Este desbalance (aprox. 3.4% de fallas) simula fielmente las condiciones reales de una planta de manufactura de alta disponibilidad, validando la necesidad de técnicas de Aprendizaje Automático capaces de gestionar clases desbalanceadas.

3. Justificación

Relevancia Práctica y Económica:

La implementación de este sistema impacta directamente en la eficiencia operativa general (OEE). Al predecir fallas antes de que ocurran, se reducen los costos asociados a tiempos de inactividad, se evita el desperdicio de materia prima por procesos defectuosos y se aumenta la seguridad de los operarios al reducir interacciones con maquinaria inestable.

Justificación del uso de Inteligencia Artificial:

Predecir fallas en una fresadora CNC es un desafío complejo porque involucra interacciones altamente no lineales entre múltiples variables termodinámicas y dinámicas (por ejemplo, cómo el desgaste de la herramienta afecta la temperatura bajo diferentes niveles de torque). Los sistemas basados en reglas tradicionales no pueden modelar estas interacciones ocultas, haciendo indispensable el uso del Aprendizaje Automático.

El modelo que se seleccionó fue Random Forest, después se comparó con Regresión Logística y SVM (Máquina de Soporte Vectorial).

4. Procedimiento

4.1 Importar las librerías necesarias para el funcionamiento correcto del proyecto:

```
# Librerías generales de datos y visualización
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Librerías de Machine Learning (Scikit-Learn)
from sklearn.metrics import recall_score, precision_score, f1_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

4.2 Carga y exploración de datos

Primeramente subimos el archivo para poder manipularlos, y solo tomamos una pequeña muestra, y verificamos el balance de clases del dataset.

```
# Carga y exploración de datos
archivo = 'ai4i2020.csv'
df = pd.read_csv(archivo)

print("Vista previa de los datos")
display(df.head())

print("--- INFORMACIÓN DEL DATASET ---")
print(f"Fuente: UCI Machine Learning Repository")
print(f"Tamaño (Filas, Columnas): {df.shape}")
print("\nTipos de Variables:")
print(df.dtypes)

print("\n--- DISTRIBUCIÓN DE FALLAS ---")
conteo = df['Machine failure'].value_counts()
print(f"Máquinas OK (0): {conteo[0]}")
print(f"Máquinas con Falla (1): {conteo[1]}")
```

```
--- INFORMACIÓN DEL DATASET ---
Fuente: UCI Machine Learning Repository
Tamaño (Filas, Columnas): (10000, 14)

Tipos de Variables:
UDI                int64
Product ID         object
Type               object
Air temperature [K] float64
Process temperature [K] float64
Rotational speed [rpm] int64
Torque [Nm]        float64
Tool wear [min]    int64
Machine failure     int64
TWF                int64
HDF                int64
PWF                int64
OSF                int64
RNF                int64
dtype: object

--- DISTRIBUCIÓN DE FALLAS ---
Máquinas OK (0): 9661
Máquinas con Falla (1): 339
```

Vista previa de los datos

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	0	0	0	0	0

4.3 Limpieza y preparación

- Primeramente eliminamos variables que no son relevantes para el modelo, Unique Identifier (UDI) y Product ID son simples identificadores, no son características físicas ni operativas, no contienen patrones generalizables y pueden confundir al algoritmo.
- La columna Type tiene valores de texto categóricos: Low, Medium y High. Por ello las convertimos en variables dummy.
- Al separar en variables "x" y "y" dropeamos variables para evitar que el modelo haga trampa (Data Leakage). Machine Failure dice directamente lo que buscamos, hay que evitar que el modelo conozca esta variable. Las fallas específicas ('TWF', 'HDF', 'PWF', 'OSF', 'RNF') son subcategorías del fallo principal, si alguna tiene 1, automáticamente Machine Failure será 1 también.

Se usa stratify porque los datos están muy desbalanceados, se tiene que usar para que se mantenga la misma proporción en el train y en el test. Y aplicamos fit transform a xtrain para saber la media y desviación estándar de los datos de entrenamiento y luego se escalan. En test solo se aplica la media y desviación estándar que se aprendió del entrenamiento a los datos nuevos.

```
# Eliminamos identificadores que no aportan valor
df_clean = df.drop(columns=['UDI', 'Product ID'])

# Convertimos variables categóricas en dummy
df_clean = pd.get_dummies(df_clean, columns=['Type'], drop_first=True)

# Separamos X (predictoras) e y (objetivo), evitando "Data Leakage"
x = df_clean.drop(columns=['Machine failure', 'TWf', 'HDF', 'PWF', 'OSF', 'RNF'])
y = df_clean['Machine failure']

# División en Train y Test (Estratificado por el desbalance)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Estandarización de los datos
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

4.4 Entrenamiento y evaluación de RF

Solo hay 7 variables predictoras, por lo que se quitaron los Id's y las dummies se crearon correctamente.

- Las fallas representan aproximadamente el 3% en el conjunto de entrenamiento y el de prueba, por lo que el modelo se evaluará de forma justa.
- Como hay un gran desbalance en los datos, puede obtener un accuracy bastante alto pero seria inutil para detectar los daños reales. Usaremos entonces **class_weight='balanced'** para que el algoritmo le preste mucha más atención a la clase minoritaria (las fallas).

Tenemos que prestar atención a la sensibilidad (recall), nos va a decir, de las máquinas que realmente iban a fallar, ¿Cuál es el porcentaje que el modelo logró detectar? y de precisión, de todas las veces que el modelo dijo que iba a fallar, ¿Cuántas veces tuvo razón?

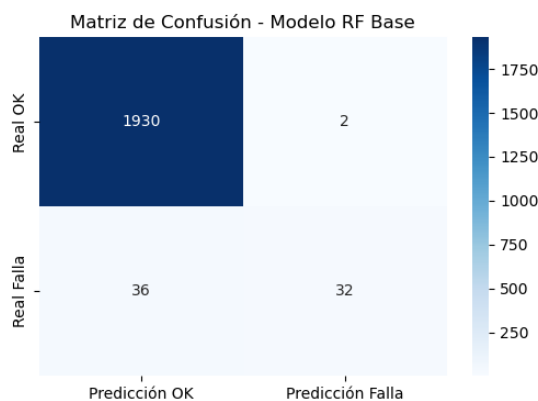
```
--- REPORTE DE CLASIFICACIÓN ---
              precision    recall  f1-score   support

     0       0.98         1.00         0.99         1932
     1       0.94         0.47         0.63           68

 accuracy                   0.98         2000
 macro avg              0.96         0.73         0.81         2000
 weighted avg           0.98         0.98         0.98         2000
```

```
# ENTRENAMIENTO Y EVALUACIÓN: Random Forest (Modelo Base)
rf_model = RandomForestClassifier(random_state=42, class_weight='balanced')
rf_model.fit(X_train_scaled, y_train)
y_pred = rf_model.predict(X_test_scaled)
print("\n - REPORTE DE CLASIFICACIÓN -")
print(classification_report(y_test, y_pred))

# Matriz de confusión del modelo optimizado
cm_opt = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,4))
sns.heatmap(cm_opt, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicción OK', 'Predicción Falla'],
            yticklabels=['Real OK', 'Real Falla'])
plt.title('Matriz de Confusión - Modelo RF Base')
plt.show()
```



Nota: Importancia de las variables

Una ventaja sustancial de Random Forest es que no es una caja negra, podemos ver directamente las variables que se consultaron para tomar las decisiones, por lo que podemos hacer Feature Importance. Lo hacemos con las columnas originales porque x train es un arreglo matemático.

Se puede observar que Torque, Rotational Speed y Tool wear dominan con un casi 75% en total. Tiene sentido, las máquinas se rompen cuando se les exige demasiada fuerza a velocidades altas, y más si la herramienta lleva muchos minutos de uso.

Air temperatur y Process Temperature tienen una importancia de alrededor del 10% cada una, el sobrecalentamiento influye en la falla pero es más una consecuencia del alto torque o la fricción que una causa principal. Las variables dummies son casi irrelevantes para el modelo, no afecta tanto la etiqueta de calidad de la fábrica, si los sensores indican que el torque y el RPM van mal, la máquina va a fallar sin importar de qué tipo sea.

```
df_importancias = pd.DataFrame({
    'Variable': X.columns,
    'Importancia': rf_model.feature_importances_
}).sort_values(by='Importancia', ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(x='Importancia', y='Variable', data=df_importancias)
plt.title('Importancia de las Variables (Random Forest Base)', fontsize=14)
plt.xlabel('Importancia Relativa (0 a 1)', fontsize=12)
plt.ylabel('Variable', fontsize=12)
plt.tight_layout()
plt.show()

##### Mostrar la tabla exacta
print("\nTabla de Importancia de Variables:")
display(df_importancias)
```

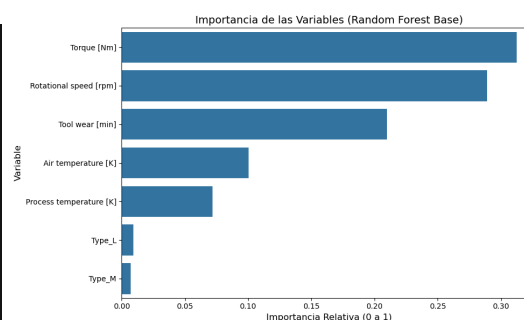


Tabla de Importancia de Variables:

	Variable	Importancia
3	Torque [Nm]	0.312612
2	Rotational speed [rpm]	0.289263
4	Tool wear [min]	0.209803
0	Air temperature [K]	0.100226
1	Process temperature [K]	0.071705
5	Type_L	0.009284
6	Type_M	0.007107

Optimización

Por lo anterior que se pudo observar, vamos a realizar una reducción de dimensionalidad eliminando las variables más relevantes. Esto va a ayudar a quitar ruido del modelo, que sea más rápido, ligero, y simple.

4.5 Comparación de modelo base y optimizado

El modelo base, tiene un recall de 47%, este logro aumentar a un 56%, ahora detecta en casi un 10% las fallas reales. El f1-score general también mejoró de 63% a 68%, aunque la precisión si bajo de 0.94 a 0.88 (tiene más falsas alarmas) el aumento en las fallas detectadas compensa. El f1 score nos confirma que el nuevo modelo es mejor y más equilibrado.

```
Nueva forma de X Optimizada: (10000, 5)
```

--- REPORTE DE CLASIFICACIÓN (MODELO OPTIMIZADO) ---				
	precision	recall	f1-score	support
0	0.98	1.00	0.99	1932
1	0.88	0.56	0.68	68
accuracy			0.98	2000
macro avg	0.93	0.78	0.84	2000
weighted avg	0.98	0.98	0.98	2000

```
# eliminamos las variables 'type' tras comprobar que su importancia es casi nula
columnas_a_eliminar = [
    'Machine failure', 'TWF', 'HOF', 'PWF', 'OSF', 'RNF',
    'Type_L', 'Type_M'
]
X_optimizada = df_clean.drop(columns=columnas_a_eliminar)
print(f"Nueva forma de X Optimizada: {X_optimizada.shape}")

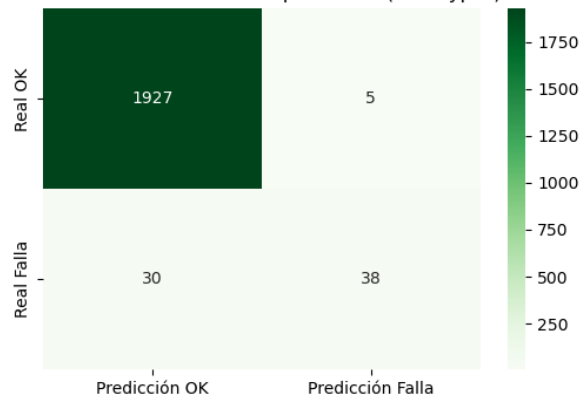
# Nueva división y escalado
X_train_opt, X_test_opt, y_train, y_test = train_test_split(
    X_optimizada, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler_opt = StandardScaler()
X_train_opt_scaled = scaler_opt.fit_transform(X_train_opt)
X_test_opt_scaled = scaler_opt.transform(X_test_opt)

# Entrenamiento del modelo optimizado
rf_model_opt = RandomForestClassifier(random_state=42, class_weight='balanced')
rf_model_opt.fit(X_train_opt_scaled, y_train)

y_pred_opt = rf_model_opt.predict(X_test_opt_scaled)
```

Matriz de Confusión - RF Optimizado (Sin "Type")



4.6 Búsqueda de Hiper-parámetros con GridSearchCV

Ahora con GridSearch podemos probar cientos de combinaciones de configuraciones para encontrar el que maximice nuestro Recall.

Con esto se logró un salto bastante grande, de un 47% con el modelo base, y un 56% quitando variables inútiles o de poca importancia, el modelo ahora detecta hasta el 75% de las fallas reales. Es decir, el modelo es capaz de anticipar 3 de 4 averías antes de que ocurran.

La precisión bajó a un 58%, por lo que ahora hay más falsas alarmas, pero esto no es necesariamente malo, es mejor que un técnico revise una máquina sana por un momento, a que una deje de funcionar sin avisar.

GridSearch encontró que una profundidad de 10 y un split de 10, el modelo evita memorizar los datos, y que solo con 50 árboles, el modelo toma mejores decisiones que con 100 o 200.

```
Mejores parámetros encontrados:
{'class_weight': 'balanced', 'max_depth': 10, 'min_samples_split': 10, 'n_estimators': 50}

--- REPORTE DE CLASIFICACIÓN (Mejor Modelo) ---
      precision    recall  f1-score   support

      0       0.99      0.98      0.99      1932
      1       0.58      0.75      0.65        68

 accuracy          0.97      2000
 macro avg       0.79      0.87      0.82      2000
weighted avg       0.98      0.97      0.97      2000
```

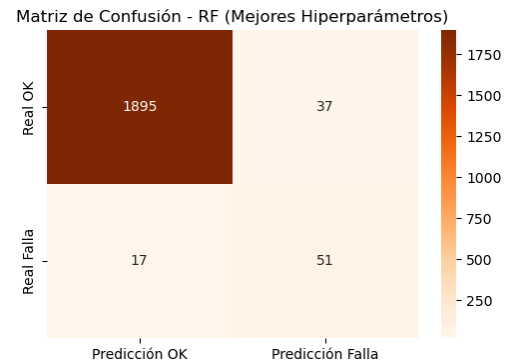
```
##### Definir la cuadrícula de parámetros a probar
# Vamos a probar diferentes cantidades de árboles y profundidades
param_grid = {
    'n_estimators': [50, 100, 200],      # Cantidad de árboles en el bosque
    'max_depth': [None, 10, 20],         # Profundidad máxima de cada árbol
    'min_samples_split': [2, 5, 10],     # Mínimo de muestras para dividir un nodo
    'class_weight': ['balanced']         # Siempre mantenemos esto para clases desbalanceadas
}

##### Inicializar el modelo base y el GridSearchCV
rf_base = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(
    estimator=rf_base,
    param_grid=param_grid,
    cv=5,                                # Validación cruzada de 5 pliegues
    scoring='recall',                    # Métrica enfocada
    n_jobs=-1                            # Núcleos del procesador para acelerar el proceso
)

##### Ejecutar la búsqueda con nuestros datos optimizados y escalados
grid_search.fit(X_train_opt_scaled, y_train)

##### Ver los resultados ganadores
print(f"Mejores parámetros encontrados:\n {grid_search.best_params_}")

##### Evaluar el modelo ganador
best_rf_model = grid_search.best_estimator_
y_pred_best = best_rf_model.predict(X_test_opt_scaled)
```



4.7 Comparación con Regresión logística y SVM (Optimizados para igualdad de condiciones)

Para finalizar, es bueno comparar con otras arquitecturas para ver cómo se desenvuelven con el mismo problema

- Regresión Logística es un modelo lineal, va a intentar separar las máquinas sanas de las que no con una línea recta.
- SVM es un modelo más complejo, si no puede hacerlo con una línea recta, puede usar kernel para doblar el espacio y encontrar separaciones complejas

```
Mejores parámetros encontrados (LR): {'C': 0.1, 'solver': 'liblinear'}
```

```
--- REPORTE DE CLASIFICACIÓN (REGRESIÓN LOGÍSTICA OPTIMIZADA) ---
      precision    recall  f1-score   support

      0       0.99      0.82      0.90      1932
      1       0.14      0.84      0.24        68

 accuracy          0.82      2000
 macro avg       0.57      0.83      0.57      2000
weighted avg       0.96      0.82      0.87      2000
```

```
# --- REGRESIÓN LOGÍSTICA (OPTIMIZADA CON GRID SEARCH) ---

# Definir la cuadrícula de parámetros
param_grid_lr = {
    'C': [0.01, 0.1, 1, 10, 100],      # C controla la regularización (inverso a la fuerza)
    'solver': ['liblinear', 'lbfgs']    # Algoritmos de optimización
}

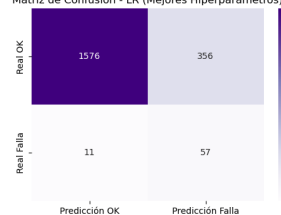
# Inicializar el modelo base y GridSearchCV
lr_base = LogisticRegression(random_state=42, class_weight='balanced', max_iter=2000)
grid_search_lr = GridSearchCV(
    estimator=lr_base,
    param_grid=param_grid_lr,
    cv=5,                                # Validación cruzada de 5 pliegues
    scoring='recall',                    # Optimizamos para Recall
    n_jobs=-1
)

# Realizar la búsqueda
grid_search_lr.fit(X_train_opt_scaled, y_train)

# Ver los resultados ganadores
print(f"Mejores parámetros encontrados (LR): {grid_search_lr.best_params_}")

# Evaluar el modelo ganador
best_lr_model = grid_search_lr.best_estimator_
y_pred_lr_best = best_lr_model.predict(X_test_opt_scaled)
```

Matriz de Confusión - LR (Mejores Hiperparámetros)

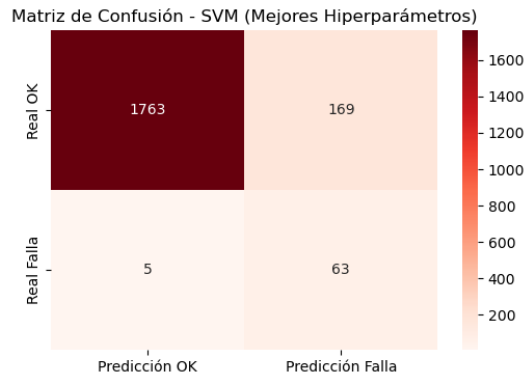


```
Mejores parámetros encontrados (SVM): {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}

--- REPORTE DE CLASIFICACIÓN (SVM OPTIMIZADO) ---
      precision    recall  f1-score   support

     0       1.00      0.91      0.95      1932
     1       0.27      0.93      0.42         68

 accuracy          0.91      2000
  macro avg       0.63      0.92      0.69      2000
 weighted avg     0.97      0.91      0.93      2000
```



5. Resultados

Regresión Logística: Aunque el recall aumentó a un 0.84, la precisión se fue al suelo con un 0.14, tiene demasiadas falsas alarmas. El modelo intentó separar con una línea pero el desgaste de una máquina es complejo (no lineal), no puede encontrar una buena frontera, además, el parámetro balanced le hace tener falsas alarmas ante la menor duda.

Support Vector Machine: Obtuvo un recall bastante bueno de 0.93, pero la precisión sigue siendo muy baja con un 0.27, seguirá teniendo demasiadas falsas alarmas al igual que Regresión Logística.

```
=== RESUMEN DE COMPARACIÓN DE MODELOS OPTIMIZADOS ===
```

	Modelo	Recall	Precisión	F1-Score	Accuracy
0	Random Forest (Campeón)	0.750	0.580	0.654	0.973
2	SVM	0.926	0.272	0.420	0.913
1	Regresión Logística	0.838	0.138	0.237	0.816

Modelo Ganador - Bootstrapping

Aunque SVM logra un recall más alto, Random Forest Optimizado sigue siendo el mejor. Si se usa SVM lanzará muchas falsas alarmas de máquinas que si están en buen estado. Eventualmente, un equipo real se llegaría a hartar y probablemente ignoraba el modelo.

En cambio Random Forest tiene un equilibrio mejor para una aplicación real, es mucho más certero cuando habla, más de la mitad de sus alarmas son reales y se tiene un mayor nivel de confianza que con SVM.

Para evaluar la robustez del modelo, aplicamos Bootstrapping con 1000 iteraciones sobre el conjunto de pruebas para calcular intervalos de confianza al 95% y asegurarnos que las métricas no son producto del azar.

6. Interpretación de Resultados

- **Recall 0.750:** Podemos afirmar con un 95% de confianza que, si este modelo se implementa en producción con datos similares, detectará consistentemente entre el 64.5% y el 84.5% de las fallas reales.
- **Precisión 0.579:** La precisión esperada fluctuará entre 47% y 68%. Entonces al menos la mitad de las alarmas serán verdaderas, demostrando que es un modelo confiable para el equipo de mantenimiento.
- **F1-Score 0.652:** La estabilidad de este intervalo confirma que el equilibrio del modelo es sólido frente a variaciones aleatorias en los datos de entrada (no sufre varianza extrema).

El Random Forest optimizado posee una varianza controlada y predecible. Los resultados son rangos matemáticamente respaldados que justifican y aseguran la viabilidad financiera y operativa del modelo para su despliegue industrial.

```
CÁLCULO DE INCERTIDUMBRE E INTERVALOS DE CONFIANZA (95%)  
  
--- RESULTADOS ESTADÍSTICOS PARA LA CLASE 1 (FALLAS) ---  
Recall:    0.750 (Intervalo 95%: [0.645, 0.845])  
Precision: 0.579 (Intervalo 95%: [0.471, 0.682])  
F1-Score:  0.652 (Intervalo 95%: [0.559, 0.737])
```

7. Conclusiones

1. Sesgos en los Datos (Data Biases)

Existen varios sesgos del dataset, el primero siendo el fuerte sesgo de entorno, ya que se asumen condiciones perfectamente controladas, si este modelo se despliega en una fábrica real, con diferentes parámetros (humedad, altitud, fluctuaciones de temperatura ambiente) las variables térmicas del proceso perderían su calibración inicial.

Además este dataset carece de variables como vibraciones externas, fallos temporales de lectura de los sensores o desgaste. Y por último, hay un punto ciego en los estados intermedios, donde una máquina aún opera pero ya presenta un desgaste, solamente distingue entre fallas catastróficas y operaciones ideales.

2. Limitaciones del Modelo

El principal desafío operativo de este modelo es la fatiga de alarmas, ya que se sacrificó la precisión general para priorizar la detección de averías. En la práctica 4 de cada 10 alertas serán falsas alarmas, lo que podría mermar la confianza de los técnicos. Tampoco hay una ventana de tiempo (Time-to-failure). Es un modelo de clasificación binaria que evalúa el estado del sensor en tiempo real, no puede predecir en cuantas horas ocurrirá el colapso, solamente da la advertencia.

3. Viabilidad de Despliegue

El despliegue de este modelo es bastante viable, siempre y cuando tenga un esquema de asistencia y supervisión humana. El costo operativo por reemplazar piezas destrozadas y detener la línea de producción supera por mucho la pérdida de diez minutos de un técnico revisando una máquina sana por una falsa alarma. Random Forest Optimizado demostró una capacidad estadística para anticipar 3 de 4 alarmas de forma consistente.

Lo recomendado sería desplegar inicialmente paralelamente (Shadow Mode) por un mes, para calibrar la sensibilidad de las alertas y entrenar al personal de mantenimiento antes de darle al modelo el control para detener a las máquinas.

Bibliografía:

UCI Machine Learning Repository. (n.d.).

<https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset>